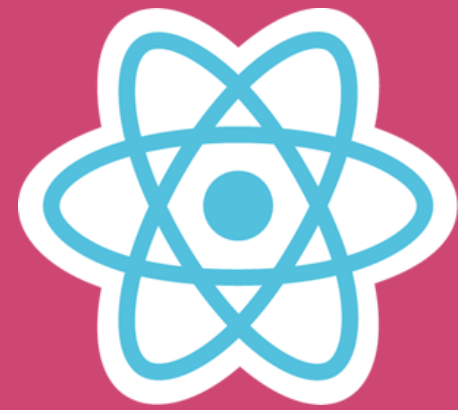
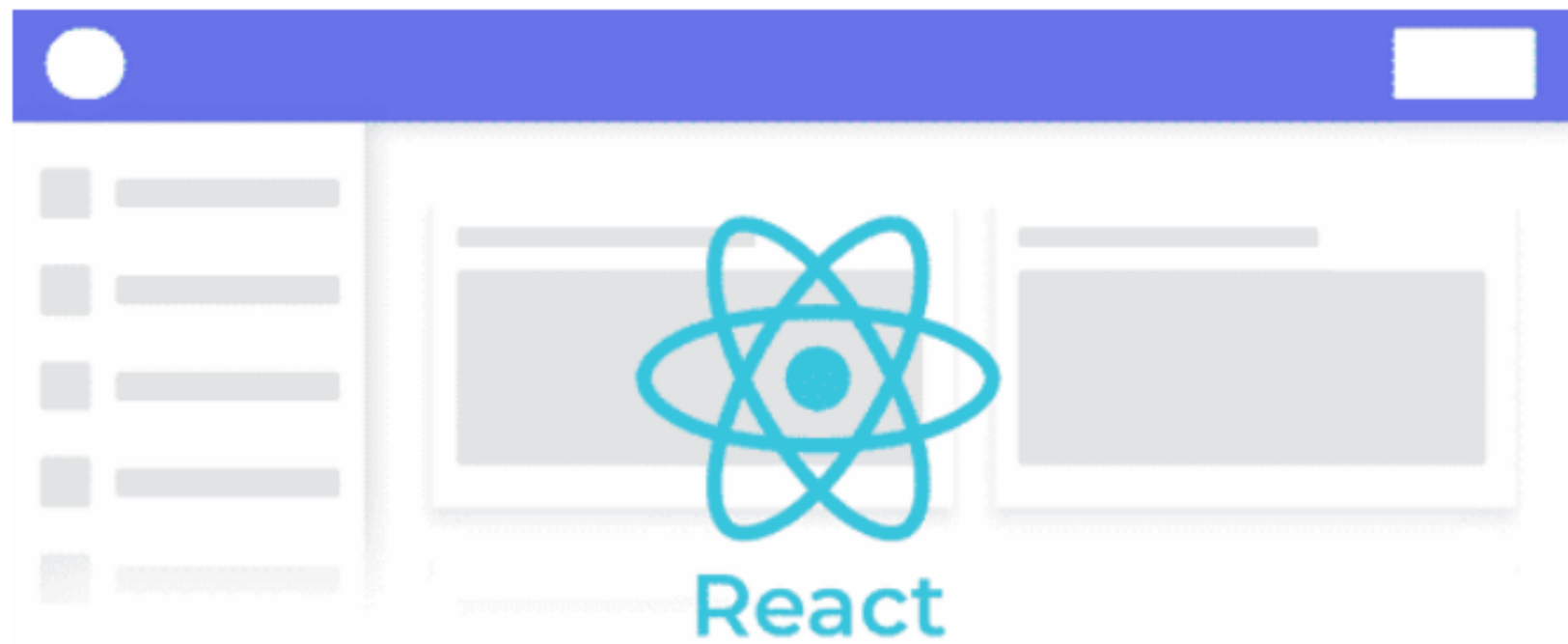


devtown



React Hooks

CHEAT SHEET 01



React Hooks - Overview

React Hooks are in-built functions that allow React developers to use state and lifecycle methods inside functional components.

- ✔ Improved code reusability.
- ✔ Focus on logic than setup
- ✔ Custom hooks to reuse logic

useState

Declaring state

```
const [name, setName] = useState("name")
```

Using & Updating state

```
<div>
  Hey ${name}, Welcome!

  <div>
    <button onClick={() =>
      setName(newValue)}>
      New name
    </button>
  </div>
</div>
```

useState is used to implement states in functional components.

useState returns an array with two elements

first one is state value

second is function to update the state value

updating the state, with a button click

useEffect

Declaring useEffect

```
useEffect(()=> {
  //your logic here, eg: API call, or event listeners
  setName("name from useEffect");
}, [ ]);
```

Clean up in useEffect

```
useEffect( => {
  // your logic here, eg: API call, or event listeners
  setName("name from useEffect");
  // return a clean up function
  return () => setName("");
}, [ ]);
```

useEffect is used to implement side effects in your react components. The terms side effects can be your API call or some logical computations.

Recall lifecycle methods from class components, useEffect hook handles our logic as same as lifecycle methods.

Clean up is really good practice, and useEffect provides an easy way to perform your clean up logic. Just return your clean up logic as a function from useEffect and react will take care of it.

Note: When ever useEffect is invoked, it executes the return/clean up statement first then your side effect logic.

empty array -> []

This means that the useEffect doesn't have any dependency it will executed only once when your component mounts.

React Hooks

CHEAT SHEET 02

With dependency in useEffect

```
useEffect( => {  
  //your logic here, eg: API call or event listners  
  
  setName("name from useEffect");  
  
  // return a clean up fucntion  
  
  return () => setName("");  
} [ name ]);
```

The dependency list can be passed as an array as the second argument.

dependency array -> [dependencylist]

This means that the useEffect will execute only when there is a change in the dependencies provided.

useReducer

Creating a reducer/state

```
// init function to reset state object  
function init(initialCount) {  
  return {count: initialCount};  
}  
  
// reducer function that decoupled and grouped our state  
function reducer(state, action){  
  switch (action.type){  
    case 'increment':  
      return{count: state.count + 1};  
    case 'decrement':  
      return{count: state.count - 1};  
    case 'reset':  
      return init(action.payload);  
    default:  
      throw new Error();  
  }  
}
```

useReducer in react is an alternative to useState, It is an efficient placeholder to store our state. Also, it provides you more control to manage the state.

The reducer is nothing but our complex state divided based on certain condition.

using useReducer hook

```
const Counter = ({initialCount}) = >  
  // calling useReducer hook to get access to the state tree  
  const [state, dispatch] = useReducer(reducer, initialCount, init);  
  return (  
    <>  
      Count: {state.count}  
      /* calling dispatch function returned from the useReducer  
       hook to dispatch actions. */  
    <button  
      onClick={() => dispatch({type: reset, payload: initialCount})}>  
      Reset  
    </button>  
    <button onClick={() => dispatch({type: decrement })}>  
    <button onClick={() => dispatch({type: increment })}>+  
    </>  
  );  
}
```

The useReducer hook accepts 3 arguments

reducer: our state tree
initialCount: initial state value, if any
init: function to reset the state tree.

useReducer hook returns the state tree and dispatch function, state tree is used to access the values and dispatch function is used to dispatch changes.

useCallback

Declaring useCallback

```
const handleClick = useCallback(() => {  
  // handle the click event  
  
  // assuming this function has some event listner  
}, []);
```

You can pass a function to useCallback and it will return the memoized version of it and you can use the memoized function in your components.

useCallback in react is used to remember a something, it might be a function or some variable. useCallback will return a memoized version of that.

Using function in JSX

```
return onClick={handleClick}>  
  
  Click Me!  
  
</button>
```

Case study

It really useful because whenever our component re-renders it creates those functions and variable again, this practice is not tolerable if we are assigning event listeners, useCallback will make sure that it doesn't re-render along with the component.

React Hooks

CHEAT SHEET 03

useRef

Declaring useRef

```
const element = useRef(null); // null or initial value
```

useRef in react is used to directly access the DOM and perform updates or any other operation. And similar to useCallback it returns the same states regardless of re-renders.

Using useRef in functions to access DOM directly

```
const handleRef = () => {  
  element.current.value; // get or add value for the element  
  element.current.focus(); // focus the element  
};
```

.current property refers to current element.

.value: will hold the current value of the referred element.

.focus: is used to focus the element.

linking the element to useRef object

```
<textarea ref={element} rows="5" cols="33" />
```

To refer an element, just pass the object that was returned from useRef to an element's ref attribute. And react will provide access for that.